



Inteligencia Artificial
Grado en Ingeniería Informática en Sistemas de Información
ENSEÑANZAS PRÁCTICAS Y DE DESARROLLO
EPD 1: Herramientas para Machine Learning

Objetivos

- Introducción al lenguaje Python y al entorno de desarrollo JupyterLab

Conceptos

Python es un lenguaje de programación interpretado, de tipado dinámico y multiparadigma (soporta programación orientada a objetos, programación imperativa y programación funcional). Podemos destacar las siguientes características:

- Usa palabras en lugar de operadores lógicos: “and”, “or”, “not”, en vez de “&&”, “||”, “!”.
- Usa “indentaciones” o sangrado para marcar bloques de código en vez de las comunes llaves “{}” de otros lenguajes. Es muy importante controlar los espacios o tabulaciones que se usan y no mezclarlos. Se recomienda usar tabulaciones.
- Se recomienda escribir cada instrucción en una sola línea, si en algún caso se debe separar en varias líneas se puede usar “\” al final de la línea.
- Para comentarios cortos se usa “#”. Para comentarios largos se usa “ """ ” al inicio y al final.

Existen varios IDEs y editores de código para trabajar con Python: PyCharm, Spyder, Jupyter Notebook, Atom, etc. En este curso usaremos JupyterLab, que ejecutará Python a través de un entorno virtual Conda preconfigurado con todas las librerías y dependencias necesarias para el desarrollo de la asignatura.

JupyterLab es un entorno interactivo que facilita el trabajo con datos, código, y documentación de manera simultánea. Es muy utilizado en ciencia de datos, aprendizaje automático e investigación, ya que permite escribir código, ejecutarlo, visualizar resultados y documentar el proceso de una forma integrada y simple. A diferencia de un entorno de desarrollo más tradicional, JupyterLab es especialmente útil para proyectos que requieren experimentar con datos de manera interactiva y realizar un análisis paso a paso.

El funcionamiento de JupyterLab está basado en notebooks, que son documentos interactivos donde se pueden incluir fragmentos de código, gráficos, texto (utilizando Markdown), ecuaciones matemáticas y más. Los notebooks están organizados en celdas, donde cada celda puede contener diferentes tipos de contenido. Por ejemplo, puedes tener una celda con código en Python que calcule estadísticas de un conjunto de datos y una celda justo debajo explicando los resultados usando Markdown.

Introducción a Python

Librerías más comunes de Python son:

- Pandas: Permite crear Series de datos (una dimensión) y Dataframes (dos dimensiones). Es muy útil y sencilla de usar.
- Numpy: Permite crear una estructura universal de datos como vectores y matrices. Además, posee funciones matemáticas de alto nivel.
- Matplotlib: Procesa y genera gráficos de gran calidad.
- Seaborn: Librería basada en matplotlib. Visualización de datos estadísticos.
- Scikit-Learn o Sklearn: Incluye múltiples técnicas y algoritmos de machine learning y análisis de datos.
- SciPy: Crea rutinas numéricas con estructura de datos. Posee funciones como: optimización, estadística o álgebra lineal.
- Tensorflow: Se suele usar en problemas de Deep Learning. Se basa en la codificación de un grafo compuesto por nodos que son operaciones matemáticas, junto con aristas que representan los tensores.
- Keras: Se suele usar en problemas de Deep Learning.

Las librerías se importarán al inicio de cada script mediante la sentencia: `import nombreLibreria` . Aparecerá una bombilla roja en el editor para instalar la librería. Si se le quiere dar un nombre a la librería al importarla como por ejemplo: `import numpy as np` , para hacer referencia a esa librería usaremos `np.funcion(...)` . Por ejemplo: `np.divide(...)`.



Ejercicios

EJ1. Define tres variables con los siguientes valores: $a = 1.5$, $b = 3.4$, $c = 5.2$. Calcula el valor de d según la siguiente fórmula:

$$d = \frac{a}{\frac{b}{c^a} - \frac{c}{b^a}}$$

EJ2. En un Congreso Internacional de Matemáticas se votó como la fórmula más bella $e^{i\pi} = -1$. Compruebe el resultado de esta fórmula implementándola en Python. Conviene utilizar la constante `e` y `pi`, y puede que necesites redondear con la función `round`.

EJ3. Compara el tiempo empleado para sumar dos matrices de números aleatorios de dimensión 60x60 con y sin vectorización. Para medir el tiempo haga llamadas a la función `time` al principio y al final de las operaciones respectivamente.

EJ4. Multiplica los elementos de una matriz aleatoria de dimensión 10x10 por su número de fila. Hágalo de tres maneras distintas: utilizando dos bucles, un bucle y finalmente vectorizando. Compare los tiempos utilizando una matriz de mayor dimensión (1000x1000).

EJ5. Multiplica los elementos de una matriz aleatoria de dimensión 10x10 por los elementos de otra matriz aleatoria de dimensión 10x10. Hágalo de tres maneras distintas: utilizando dos bucles, un bucle y finalmente vectorizando. Compare los tiempos utilizando una matriz de mayor dimensión (1000x1000).

EJ6. Implementa una función que, dado un valor del radio de una circunferencia, devuelva el diámetro, la longitud y el área. Considere que el radio es 5. Además, implementar una función que muestre la parábola $y = x^2$ entre -10 y 10.

EJ7. Resuelva el siguiente sistema de ecuaciones lineales con arrays y usando la librería `numpy`. Hágalo de dos maneras distintas: con la función `np.linalg.inv()` para conseguir la inversa de la matriz y, la segunda, simplemente con la función `np.linalg.solve()`

$$\begin{aligned}8x + 3y - 2z &= 9 \\ -4x + 7y + 5z &= 15 \\ 3x + 4y - 12z &= 35\end{aligned}$$

EJ8. Implemente una función usando la librería `numpy` que compruebe si una matriz pasada por parámetro es mágica. una matriz es mágica si las filas, columnas y diagonales suman lo mismo. A continuación, revise la función proporcionada `magic_square()` y compruebe los resultados.

EJ9. Genere un vector de 100 números aleatorios entre 0 y 10 usando la función `random.uniform()` de `numpy`. Representar el vector con figuras `plot` y `stairs` usando la librería `matplotlib.pyplot` etiquetando correctamente cada eje e incluyendo el nombre del gráfico.

Problemas

P1. Calcular de dos formas la siguiente fórmula: $h_{\theta} = \sum_{j=0}^n \theta_j x_j = \theta^T x$

P2. Use el "iris dataset" (disponible en los datasets de la librería `sklearn`). Estos datos corresponden a la clasificación de unas plantas en tres categorías, en función de la longitud y anchura de pétalos y sépalos (<https://archive.ics.uci.edu/ml/datasets/iris>). Se pide en primer lugar, obtener una gráfica como la que se muestra a continuación, y posteriormente crear una figura con las posibles combinaciones entre los atributos. Para este último paso puede ser útil utilizar la librería `seaborn` (que está basada en la librería `matplotlib`) y en concreto la función `pairplot()`.

